БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Учебный проект по дисциплине

«Защита от вредоносного программного обеспечения»

ЛАБОРАТОРНАЯ РАБОТА № 2

«Разработка архитектуры и технической документации на антивирусное программное обеспечение»

АНТИВИРУСНОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ДЛЯ СТОМАТОЛОГИЧЕСКОЙ КЛИНИКИ «СТОМАСТРАЖ»

Учебное обозначение: СС.00001-02 АРХ

Разработали:	Капустин Александр Дятчик Владимир Дятчик Владислав
Проверил:	_____
Версия документа:	1.0

Минск, 2026

СОДЕРЖАНИЕ

1 Введение

2 Исходные требования и принятые технические решения

2.1 Архитектура

2.2 Уточнение языка и технологического стека

2.3 Ограничения учебной версии

3 Общая архитектура приложения

3.1 Архитектурные принципы

3.2 Компоненты системы

3.3 Взаимодействие компонентов

4 Структура C++-проекта

5 Ядро сканирования

5.1 Проверка одного файла

5.2 Проверка каталога и диска

6 Механизмы обнаружения

6.1 Сигнатурный анализ

6.2 Эвристический анализ

7 Мониторинг файловой системы

8 Карантин

9 Журналирование

10 Хранение данных

11 Обновление базы сигнатур

12 Планировщик проверок

13 Пользовательский интерфейс

14 Многопоточность

15 Классы и интерфейсы C++

16 Обработка ошибок и надёжность

17 Безопасность приложения

18 Сборка и развертывание

19 Тестирование

20 Последовательность разработки

21 Возможности расширения

22 Заключение

Приложение А. Условные обозначения блок-схем

Приложение Б. Пример CMakeLists.txt

1 Введение

В рамках дисциплины «Защита от вредоносного программного обеспечения» разрабатывается учебное антивирусное программное обеспечение «СтомаСтраж». На первой лабораторной работе были определены назначение программы, функциональные требования, требования к надёжности и безопасности, состав программной документации и программа испытаний. Настоящая работа переводит эти требования на уровень архитектуры: определяет модули будущего приложения, их ответственность, интерфейсы взаимодействия, структуру данных и последовательность реализации.

Целевой средой остаются персональные компьютеры регистратуры, врачей и административных работников стоматологической клиники под управлением Windows 10/11 x64. На этих компьютерах обрабатываются сведения о пациентах, документы, диагностические изображения и данные специализированных медицинских программ. Поэтому приоритетом архитектуры является минимальное воздействие на пользовательские данные: чистые файлы не изменяются, подтверждённые угрозы изолируются в карантин, а эвристически подозрительные файлы не удаляются автоматически.

Архитектура намеренно не включает драйвер файловой системы, перехват системных вызовов ядра, эмуляцию машинного кода, гипервизор, облачную песочницу и сложный поведенческий анализ. Эти механизмы характерны для промышленных антивирусных продуктов, но выходят за рамки учебного проекта и противоречат ограничениям, установленным в первой работе.

2 Исходные требования и принятые технические решения

2.1 Архитектура

Архитектура строится непосредственно из требований технического задания и эксплуатационной документации первой лабораторной работы. Обязательными считаются следующие функции:

– быстрая проверка типовых пользовательских каталогов, полная проверка выбранных локальных дисков и выборочная проверка файла, каталога или съёмного носителя;

– вычисление SHA-256 каждого проверяемого файла и поиск точного совпадения в локальной сигнатурной базе;

– статический эвристический анализ без выполнения подозрительного кода;

– классификация результата как CLEAN, SUSPICIOUS, THREAT, ERROR или SKIPPED;

– карантин с возможностью восстановления, без штатного безвозвратного удаления;

– контроль новых и изменённых файлов в выбранных каталогах без драйвера ядра;

– локальный журнал событий, история проверок и экспорт без включения содержимого медицинских файлов;

– обновление базы сигнатур из доверенного источника с проверкой версии и контрольной суммы;

– список исключений и расписание периодических проверок;

– работа основных функций сканирования и карантина без обязательного доступа к сети.

В программе и методике испытаний первой лабораторной уже определены тесты T01–T20. Поэтому архитектура ниже проектируется так, чтобы каждый из

этих тестов можно было однозначно связать с конкретным модулем и автоматизировать по мере реализации.

2.2 Уточнение языка и технологического стека

В первом варианте технического задания в качестве референсной реализации был указан Python 3.12 с Tkinter и watchdog. Во второй лабораторной работе вводится новое учебное требование: основная реализация должна выполняться на C++. Это является изменением технического решения, но не изменением функциональных требований к продукту.

Для дальнейшей разработки принимается C++20 как основной язык. Пользовательский интерфейс реализуется на Qt 6, данные хранятся в SQLite, мониторинг файловой системы строится на Windows API ReadDirectoryChangesW, а вычисление SHA-256 — на Windows Cryptography API: Next Generation (CNG). Python допускается только для вспомогательных сценариев: генерации тестовых наборов, подготовки пакетов обновлений или служебных скриптов. Исполняемое ядро антивируса от Python не зависит.

Компонент	Принятое решение	Причина выбора
Язык	C++20	Статическая типизация, высокая производительность, стандартная библиотека потоков и файловой системы.
GUI	Qt 6 Widgets	Готовые окна, таблицы, сигналы/слоты, удобная связь фоновых задач с интерфейсом.
Сборка	CMake	Единый проект для Visual Studio/Qt Creator, удобное подключение библиотек и тестов.
База	SQLite 3	Не требует отдельного сервера, поддерживает транзакции, подходит для локальной базы сигнатур и журнала.
Мониторинг	ReadDirectoryChangesW	Нативный Windows API для событий создания, изменения и переименования файлов.
SHA-256	Windows CNG	Встроенная криптографическая подсистема Windows, не требует отдельной криптобиблиотеки.
Тесты	GoogleTest или Catch2	Модульные и интеграционные тесты C++.

Компонент	Принятое решение	Причина выбора
Вспомогательный Python	необязателен	Только подготовка тестовых данных и пакетов; не входит в рабочее ядро.

2.3 Ограничения учебной версии

– нет драйвера ядра и перехвата доступа к файлу до его открытия приложением;

– нет анализа процессов в реальном времени и блокировки вредоносного процесса после запуска;

– нет эмуляции PE-кода, песочницы и динамического исполнения подозрительных объектов;

– нет сетевого IDS/IPS и анализа сетевого трафика;

– нет облачной отправки файлов или медицинских данных;

– сигнатурный механизм версии 1.0 основан на точном SHA-256 и поэтому не обнаруживает модифицированный образец без новой записи;

– эвристика является объяснимой статической системой баллов и допускает ложные срабатывания; поэтому её результат не вызывает автоматического удаления.

3 Общая архитектура приложения

3.1 Архитектурные принципы

«СтомаСтраж» проектируется как модульное настольное приложение. Интерфейс пользователя отделён от бизнес-логики, а логика обнаружения — от хранения данных и Windows-специфичных операций. Это позволяет тестировать модули независимо и заменять отдельные реализации без переписывания всей программы.

– UI не выполняет прямые SQL-запросы, вычисление хеша или операции карантина;

– Scanner не знает деталей Qt-интерфейса и возвращает структурированный ScanResult;

– доступ к сигнатурам и журналу инкапсулирован в DatabaseManager/репозиториях;

– Windows API изолируется в HashService, FileSystemMonitor и файловых утилитах;

– длительные операции выполняются не в GUI-потоке;

– все действия, изменяющие файлы или базу, сопровождаются журналированием и проверкой результата.



Рисунок 1 — Компонентная архитектура приложения

3.2 Компоненты системы

Компонент	Ответственность
MainWindow / представления Qt	Отображение состояния защиты, запуск проверок, карантин, журнал, настройки, расписание.
ApplicationController	Связывает UI и прикладные сервисы, хранит состояние текущего сеанса, формирует команды.

Компонент	Ответственность
ScanManager	Создаёт задания сканирования, управляет очередью и рабочими потоками, прогрессом и отменой.
Scanner	Проверяет один файл: исключения, SHA-256, сигнатуры, эвристика, результат, карантин по политике.
SignatureEngine	Выполняет точный поиск SHA-256 в базе и возвращает метаданные угрозы.
HeuristicEngine	Выполняет статические правила и формирует score/reasons.
FileSystemMonitor	Получает события ReadDirectoryChangesW и передаёт стабильные файлы в ScanManager.
QuarantineManager	Создаёт контейнер карантина, хранит метаданные, восстанавливает файл.
UpdateManager	Проверяет пакет обновления и атомарно заменяет сигнатурную базу.
SchedulerManager	Запускает запланированные проверки и учитывает пропущенное время после старта приложения.
ExclusionManager	Нормализует пути и проверяет, относится ли объект к исключениям.
DatabaseManager	Соединение SQLite, миграции схемы, транзакции, репозитории.
EventLogger	Пользовательский журнал в SQLite и технический лог приложения.
ConfigManager	Настройки интерфейса, лимиты, пути и параметры, не требующие отдельной таблицы.

3.3 Взаимодействие компонентов

Пользователь запускает проверку через MainWindow. ApplicationController формирует ScanRequest и передаёт его в ScanManager. ScanManager перечисляет целевые объекты и раздаёт файлы worker-потокам. Каждый worker вызывает Scanner. Scanner последовательно обращается к ExclusionManager, HashService, SignatureEngine и HeuristicEngine. При обнаружении угрозы QuarantineManager выполняет изоляцию согласно политике. Итоговый ScanResult сохраняется через DatabaseManager и передаётся в UI сигналом Qt. Таким образом, интерфейс получает готовые результаты и не содержит кода обнаружения.

4 Структура C++-проекта

Предлагаемая структура проекта разделяет публичные интерфейсы, реализации, ресурсы и тесты:

```
StomaStrazh/
├── CMakeLists.txt
├── cmake/
├── include/stomastorazh/
│   ├── core/Scanner.h
│   ├── core/ScanTypes.h
│   ├── detection/SignatureEngine.h
│   ├── detection/HeuristicEngine.h
│   ├── monitoring/FileSystemMonitor.h
│   ├── quarantine/QuarantineManager.h
│   ├── storage/DatabaseManager.h
│   ├── update/UpdateManager.h
│   ├── scheduler/SchedulerManager.h
│   ├── logging/EventLogger.h
│   └── config/ConfigManager.h
├── src/
│   ├── main.cpp
│   ├── app/ApplicationController.cpp
│   ├── core/Scanner.cpp
│   ├── core/ScanManager.cpp
│   ├── detection/SignatureEngine.cpp
│   ├── detection/HeuristicEngine.cpp
│   ├── security/HashService.cpp
│   ├── monitoring/FileSystemMonitor_win.cpp
│   ├── quarantine/QuarantineManager.cpp
│   ├── storage/DatabaseManager.cpp
│   ├── update/UpdateManager.cpp
│   ├── scheduler/SchedulerManager.cpp
│   ├── logging/EventLogger.cpp
│   ├── config/ConfigManager.cpp
│   └── ui/
│       ├── MainWindow.cpp
│       ├── ScanPage.cpp
│       ├── QuarantinePage.cpp
│       ├── LogPage.cpp
│       └── SettingsPage.cpp
├── resources/
│   ├── icons/
│   └── defaults/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── fixtures/
├── tools/
│   ├── make_test_signatures.py
│   └── make_update_package.py
└── docs/
```

Каталог tools не участвует в работе установленного антивируса. Наличие вспомогательных Python-скриптов не нарушает требование об основной реализации на C++, так как они используются только разработчиками.

5 Ядро сканирования

5.1 Проверка одного файла

Единицей проверки является файл. Scanner получает путь и контекст проверки. Путь передаётся как std::filesystem::path, чтобы корректно работать с

Unicode-путями Windows. Перед чтением выполняется нормализация пути и проверка исключений.

1) Нормализовать путь и проверить список исключений. Для исключённого объекта вернуть SKIPPED и записать событие.
2) Проверить существование файла, его тип и возможность открытия на чтение. Ошибка одного объекта не завершает всё задание.
3) Получить размер, атрибуты и время изменения. Вычислить SHA-256 потоковым чтением блоками, не загружая файл полностью в память.
4) Выполнить SignatureEngine::lookup(). При совпадении сформировать THREAT и имя угрозы.
5) При отсутствии совпадения выполнить HeuristicEngine::analyze(), получить score и список причин.
6) Если score достиг порога, вернуть SUSPICIOUS, иначе CLEAN.
7) Для THREAT применить политику карантина. Для SUSPICIOUS требуется явное действие оператора, если политика не задана отдельно.
8) Сохранить результат в журнале и вернуть ScanResult вызывающему коду.

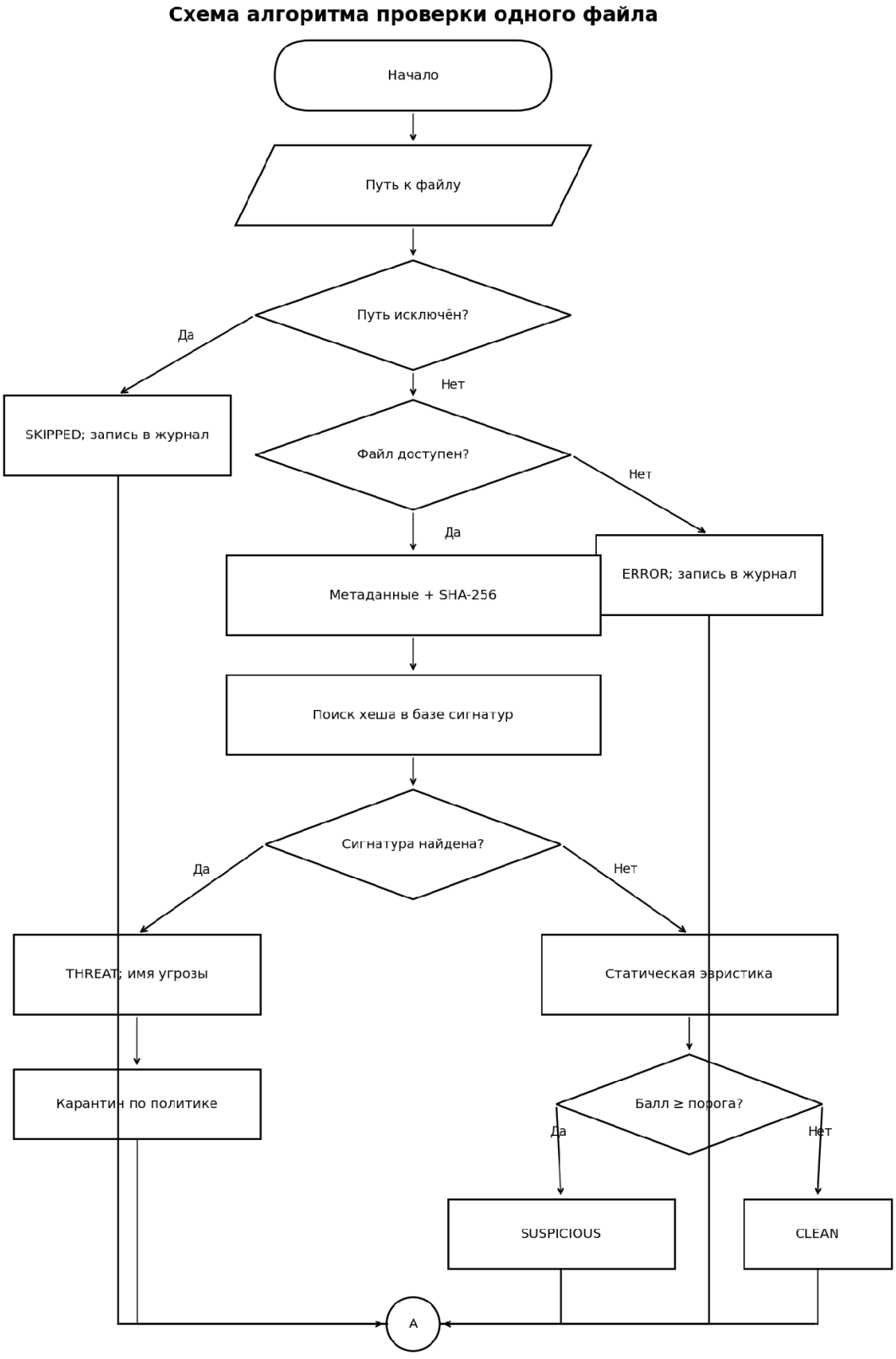


Рисунок 2 — Блок-схема проверки одного файла по ГОСТ 19.701-90

Для уменьшения гонок между вычислением хеша и последующей обработкой Scanner сохраняет исходные размер и время изменения. Если они изменились до завершения анализа, результат помечается как требующий повторной проверки. Это особенно важно для файлов, которые одновременно записываются медицинским или офисным приложением.

5.2 Проверка каталога и диска

Каталоги обходятся рекурсивно через std::filesystem. Ошибки доступа обрабатываются через std::error_code, чтобы исключение файловой системы не прерывало всю проверку. Для больших деревьев предпочтительно использовать очередь каталогов вместо глубокой рекурсии. Каждый обычный файл передаётся в ScanManager, который распределяет задания между worker-потоками.

Сканирование должно поддерживать отмену. Для C++20 удобно использовать std::stop_token: ScanManager запрашивает остановку, worker завершает текущую безопасную операцию, не берёт новые файлы и сохраняет промежуточную статистику. В журнал добавляется состояние «прервана пользователем».

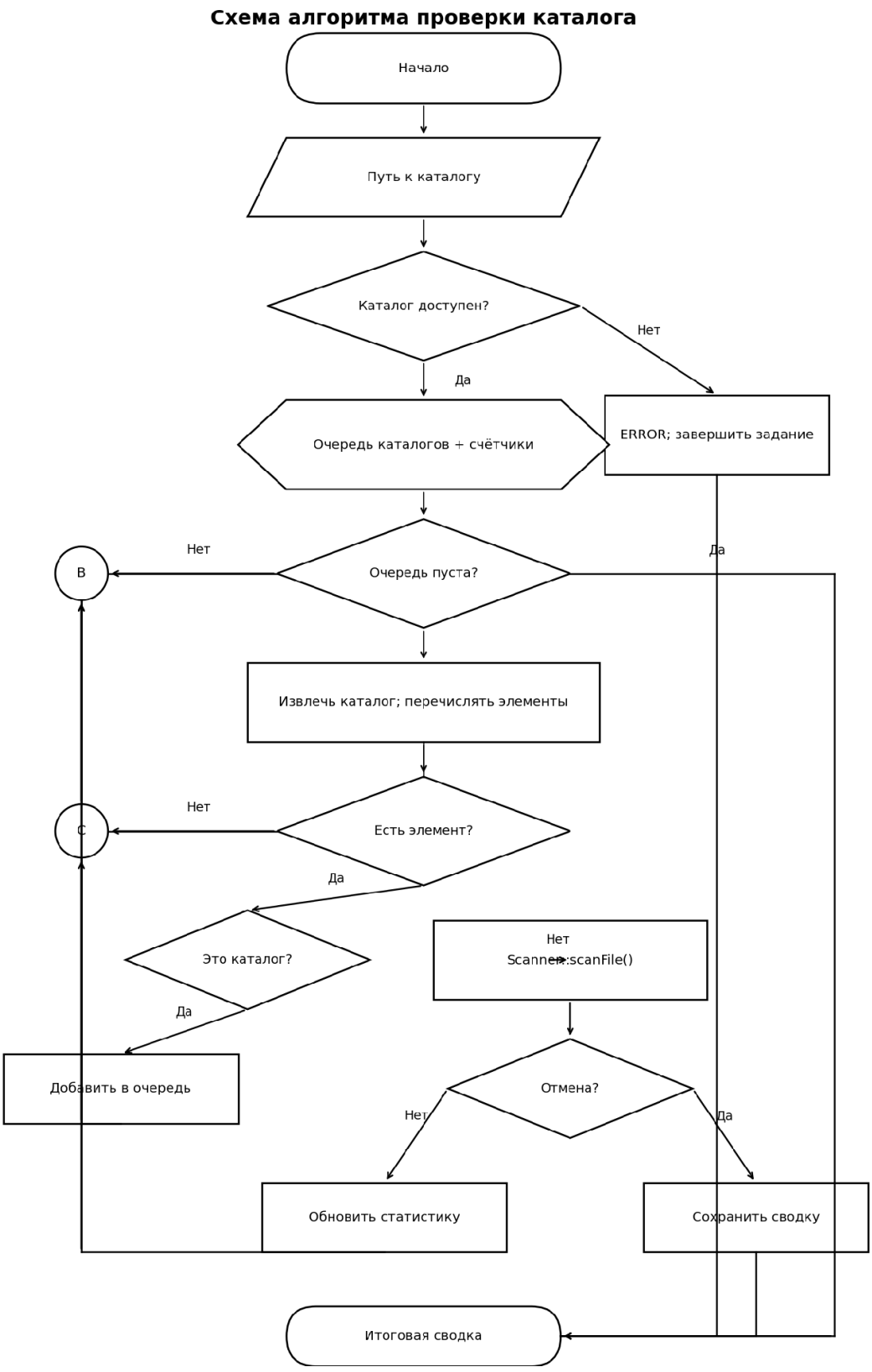


Рисунок 3 — Блок-схема проверки каталога по ГОСТ 19.701-90

6 Механизмы обнаружения

6.1 Сигнатурный анализ

Версия 1.0 использует точную сигнатуру в виде SHA-256 всего файла. HashService открывает файл только на чтение и последовательно передаёт блоки в Windows CNG. Рекомендуемый размер блока — 1–4 МиБ: он не критичен для корректности и может быть выбран по результатам тестирования. Полученная 32-байтовая сумма преобразуется в шестнадцатеричную строку нижнего регистра.

SignatureEngine выполняет параметризованный запрос по первичному ключу sha256. При совпадении возвращаются threat_name, source и версия набора сигнатур. SQLite-индекс по первичному ключу обеспечивает быстрый поиск и не требует загрузки всей базы в память.

Поле signatures	Тип	Назначение
sha256	TEXT PRIMARY KEY	64 шестнадцатеричных символа SHA-256.
threat_name	TEXT NOT NULL	Читаемое имя угрозы или тестовой сигнатуры.
source	TEXT	Источник записи: учебная база, импорт, обновление.
signature_version	INTEGER	Версия набора сигнатур.
created_at	TEXT	Дата добавления в ISO 8601.

Недостаток точного хеширования известен заранее: любое изменение файла меняет SHA-256. Поэтому механизм является первой ступенью обнаружения и дополняется эвристикой. Для учебной реализации это осознанный компромисс между простотой, воспроизводимостью тестов и реалистичной архитектурой.

6.2 Эвристический анализ

HeuristicEngine не запускает и не эмулирует исследуемый файл. Анализируется только путь, атрибуты, расширение, короткий заголовок файла и другие безопасные статические признаки. Каждое правило возвращает число

баллов и текстовую причину. Суммарная оценка сравнивается с настраиваемым порогом.

Правило	Пример веса	Условие
Двойное расширение	30	Имя вида document.pdf.exe или scan.jpg.scr.
Исполняемый файл во временном каталоге	25	Расширение .exe/.dll/.scr/.com находится в Temp/AppData\Local\Temp.
Скрытый исполняемый файл	20	Установлен FILE_ATTRIBUTE_HIDDEN и файл относится к исполняемым типам.
Несоответствие расширения и заголовка	25	Например, расширение .jpg, а первые байты указывают на PE/ZIP/PDF другого типа.
Подозрительное имя	15	Набор настраиваемых шаблонов вида invoice_copy.exe, scan_urgent.scr и т.п.
Запускоподобный тип со сменённым расширением	25	PE-заголовок MZ при неисполняемом расширении.

Для версии 1.0 разумно использовать порог 50 баллов. Значения являются конфигурационными и должны покрываться unit-тестами. Срабатывание эвристики формирует SUSPICIOUS и список причин. Автоматическое безвозвратное удаление запрещено; по умолчанию файл остаётся на месте до решения оператора.

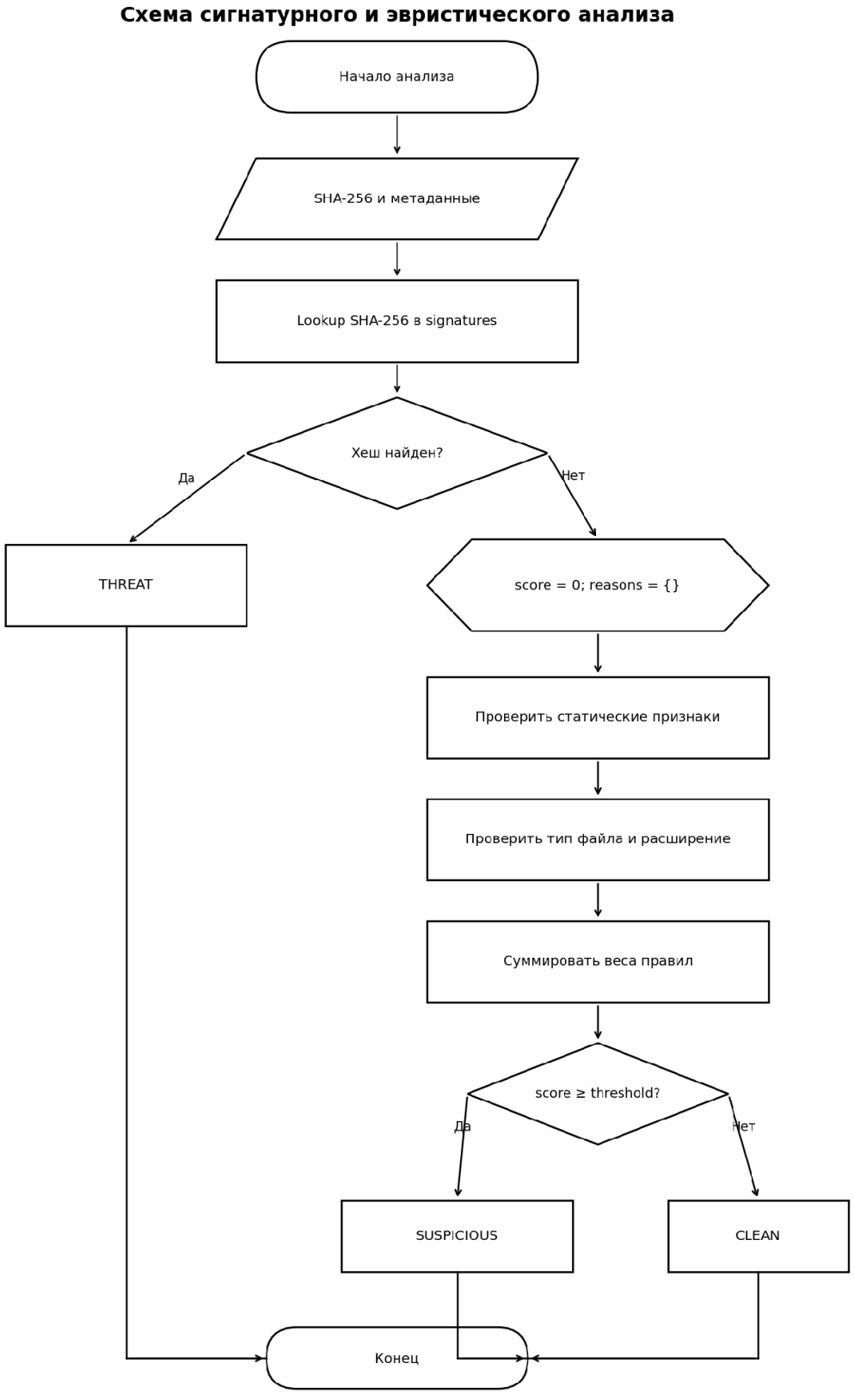


Рисунок 4 — Схема сигнатурного и эвристического анализа по ГОСТ 19.701-90

7 Мониторинг файловой системы

Мониторинг нужен для автоматической проверки новых и изменённых файлов в выбранных каталогах. Так как драйвер ядра исключён техническим заданием, применяется пользовательский механизм Windows API ReadDirectoryChangesW. Для каждого корневого каталога открывается дескриптор через CreateFileW с флагом FILE_FLAG_BACKUP_SEMANTICS. ReadDirectoryChangesW запускается асинхронно и сообщает о создании, изменении и переименовании объектов.

Обработчик событий не должен непосредственно вычислять хеш: это блокировало бы получение новых событий. Он формирует FileEvent и помещает путь в потокобезопасную EventQueue. Отдельный debounce-worker объединяет повторные события одного пути, ждёт короткий интервал стабилизации записи и только после этого ставит файл в ScanManager.

– события каталога карантина, базы и рабочих файлов самого «СтомаСтража» исключаются из мониторинга, чтобы не создать цикл;
– повторные created/modified для одного пути в коротком окне объединяются;
– если файл ещё занят процессом записи, проверка откладывается и повторяется ограниченное число раз;
– если файл удалён до проверки, событие спокойно отбрасывается с диагностической записью;
– мониторинг не гарантирует блокировку запуска файла до первого доступа; это явно остаётся ограничением учебной версии.

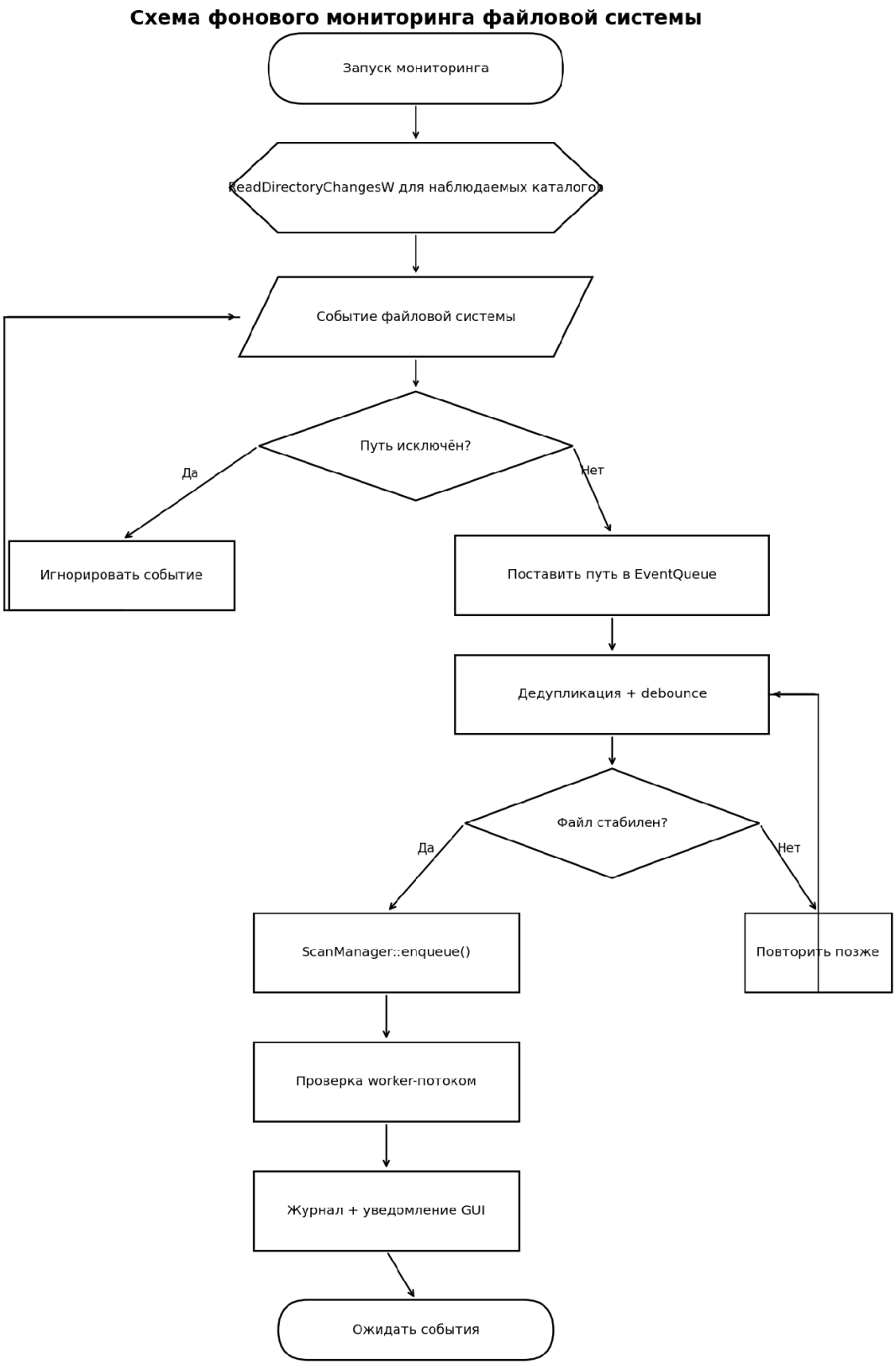


Рисунок 5 — Блок-схема фонового мониторинга по ГОСТ 19.701-90

8 Карантин

Карантин должен изолировать обнаруженный объект и одновременно сохранять возможность безопасного восстановления. Каталог карантина размещается, например, в %ProgramData%\StomaStrazh\Quarantine. При установке ему назначаются права доступа, исключающие свободное изменение обычным пользователем. Внутренние имена файлов не повторяют исходные имена и формируются как UUID с расширением .qtn.

Для объекта сохраняются исходный путь, SHA-256, имя угрозы или причина, дата помещения, имя контейнера и состояние. Содержимое файла не записывается в SQLite: база хранит только метаданные, сам объект находится в каталоге карантина.

1) Проверить, что исходный файл доступен и ещё не находится в карантине.
2) Сгенерировать случайное имя и создать временный контейнер в каталоге карантина.
3) Скопировать данные и вычислить SHA-256 копии. Если хеш отличается, временная копия удаляется, исходник остаётся на месте.
4) Открыть транзакцию SQLite и создать запись карантина со статусом PREPARED.
5) Удалить исходный файл и переименовать временный контейнер в окончательное имя. Для файла на том же томе операция переименования должна быть атомарной настолько, насколько позволяют средства ОС.
6) Изменить состояние записи на QUARANTINED и добавить событие в журнал.

Такой порядок выбран специально: исходный файл не удаляется, пока не создана и не проверена карантинная копия. Если объект находится на съёмном носителе, операция фактически становится copy-verify-delete, поэтому сбой на стадии копирования не приводит к потере исходных данных.

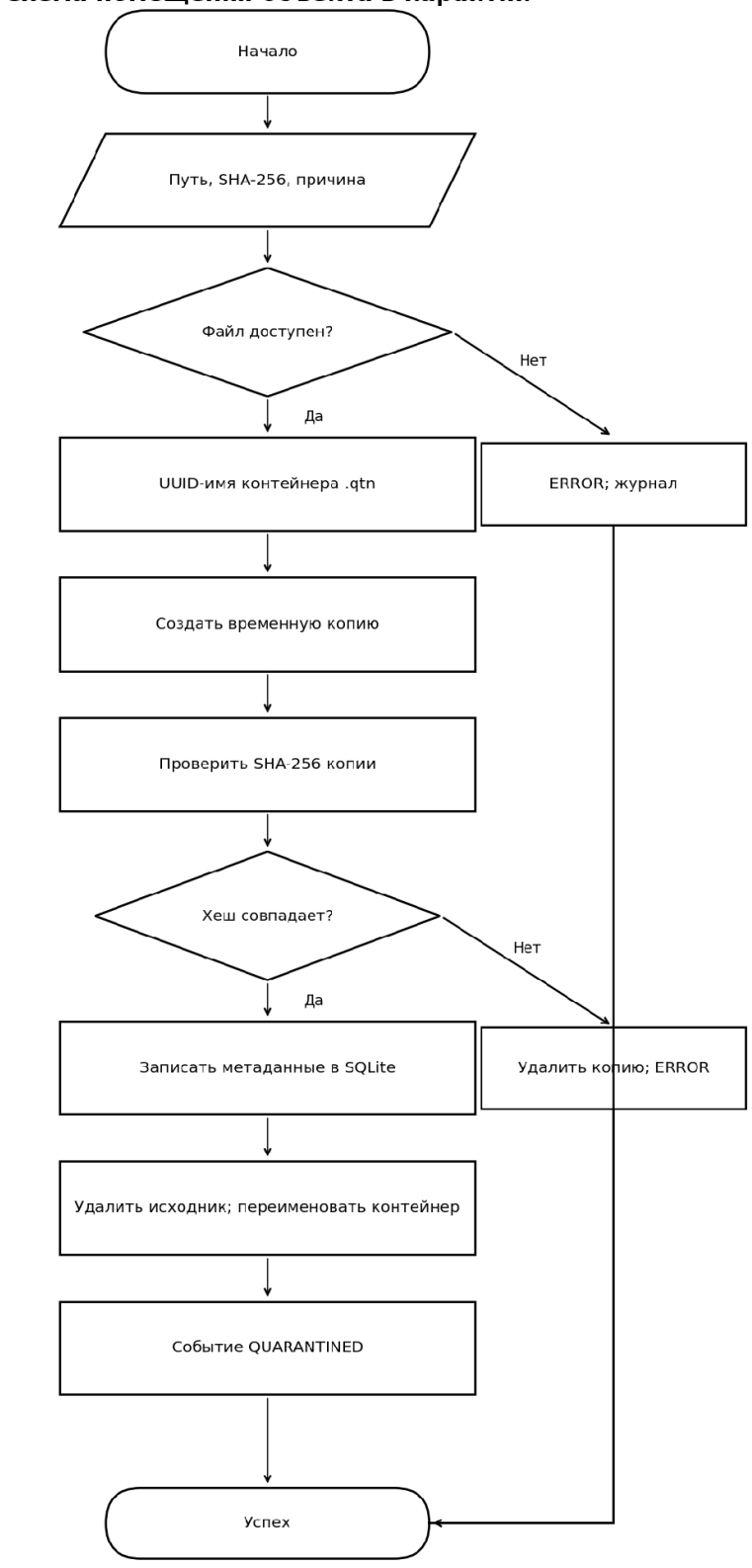


Рисунок 6 — Блок-схема помещения объекта в карантин по ГОСТ 19.701-90

Восстановление выполняется только по явному действию пользователя. Перед восстановлением сверяется контрольная сумма контейнера. Если исходный путь занят существующим файлом, оператор должен выбрать новый путь или отдельно подтвердить замену. После записи создаётся временный файл, проверяется SHA-256 и только затем выполняется окончательное переименование.

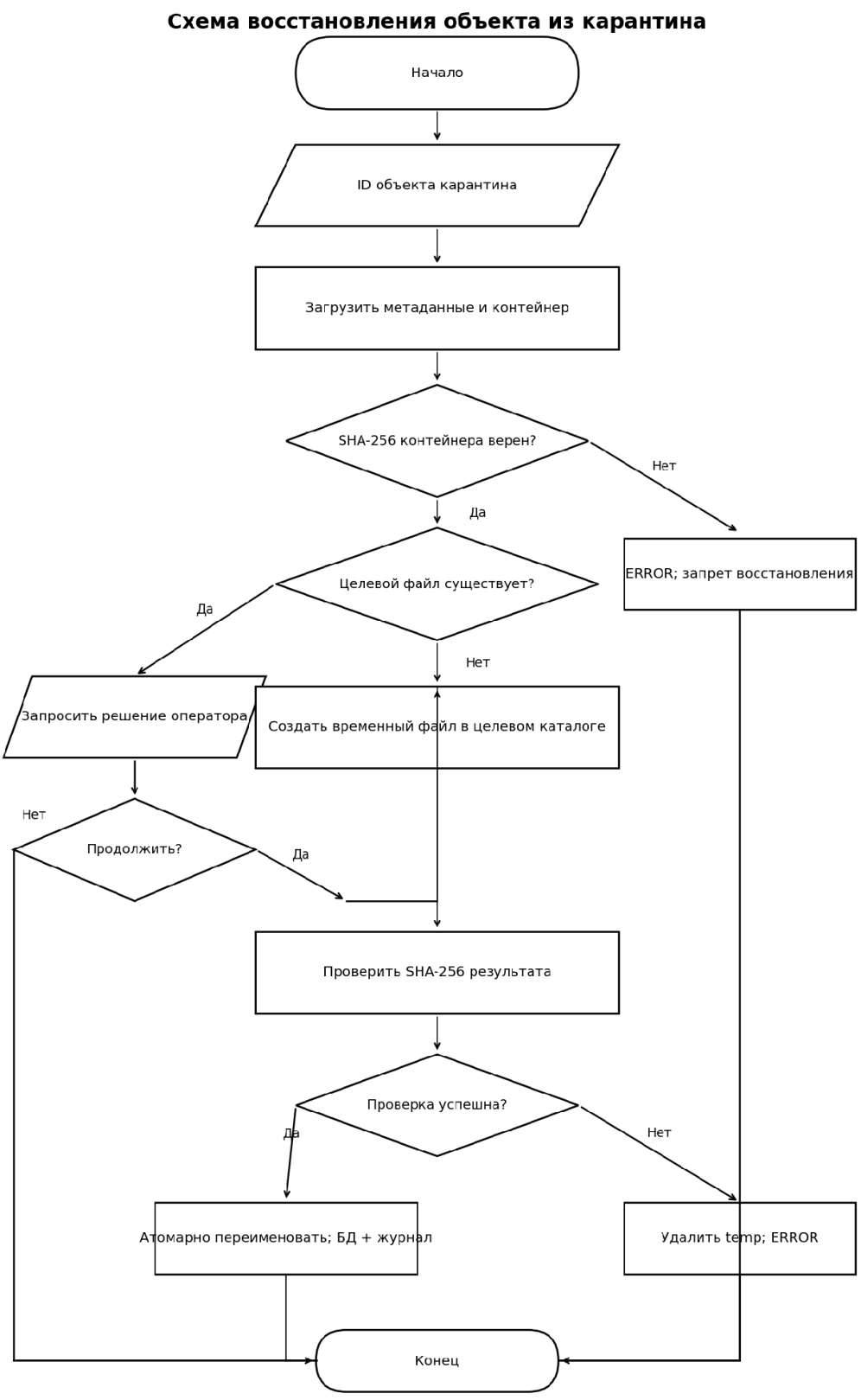


Рисунок 7 — Блок-схема восстановления из карантина по ГОСТ 19.701-90

9 Журналирование

В проекте разделяются пользовательский журнал и технический диагностический лог. Пользовательский журнал хранится в SQLite и используется в окне «Журнал». Он содержит события, которые важны для эксплуатации и испытаний. Диагностический лог предназначен разработчикам и содержит сообщения о внутренних ошибках, но не должен содержать содержимое файлов, пароли или медицинские данные.

Категория события	Примеры
SCAN	SCAN_STARTED, SCAN_FINISHED, SCAN_CANCELLED
DETECTION	THREAT_FOUND, SUSPICIOUS_FOUND, CLEAN_RESULT
QUARANTINE	QUARANTINED, RESTORED, QUARANTINE_ERROR
UPDATE	UPDATE_STARTED, UPDATE_OK, UPDATE_REJECTED
MONITOR	MONITOR_STARTED, MONITOR_STOPPED, EVENT_DROPPED
ERROR	FILE_ACCESS_ERROR, DB_ERROR, HASH_ERROR
CONFIG	EXCLUSION_ADDED, EXCLUSION_REMOVED, SETTINGS_CHANGED

Для события достаточно хранить timestamp, severity, event_code, object_path, scan_session_id, result и message. При экспорте в CSV/JSON не включается содержимое исследуемых файлов. Путь может содержать сведения о рабочем процессе, поэтому экспорт журнала также рассматривается как служебная информация.

10 Хранение данных

Основное структурированное хранилище — SQLite. Для однопользовательского настольного приложения отдельный сервер БД не нужен. Рекомендуется включить WAL-режим и сериализовать записи через одну очередь, чтобы worker-потоки не конкурировали за транзакции. Чтение сигнатур допускается из отдельных read-only соединений, если это потребуется по результатам нагрузки.

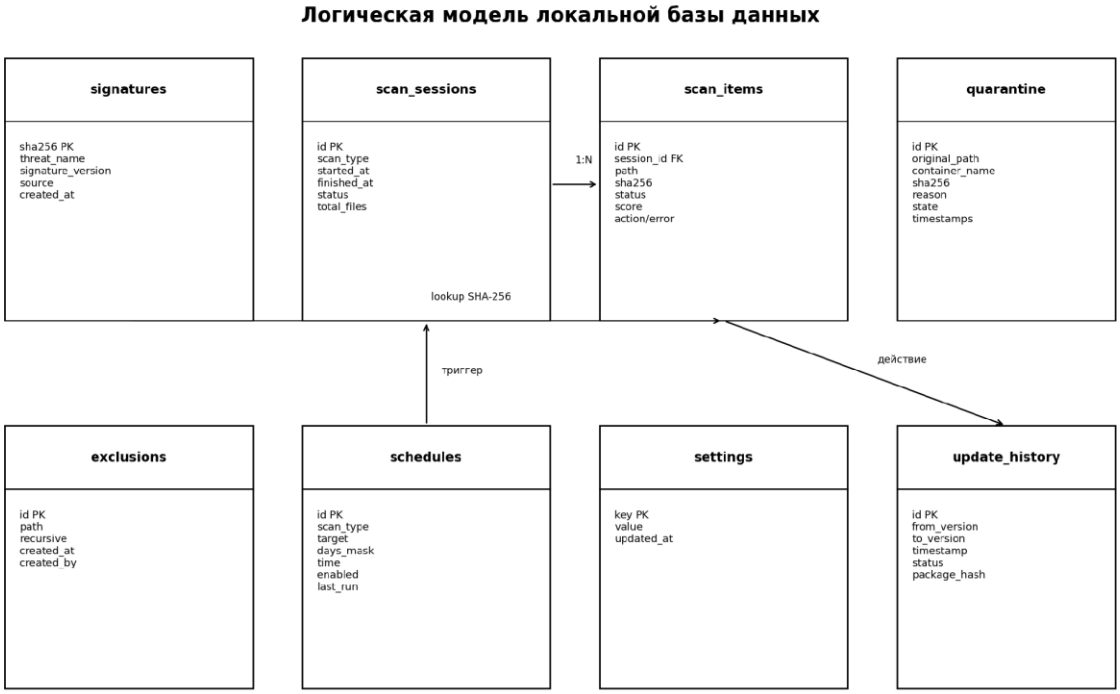


Рисунок 8 — Логическая модель локальной базы данных

Таблица	Ключевые поля	Назначение
signatures	sha256, threat_name, signature_version	Точная база сигнатур.
scan_sessions	id, type, started_at, finished_at, status	Одна пользовательская или плановая проверка.
scan_items	session_id, path, sha256, status, score, error	Результаты по отдельным файлам.
quarantine	original_path, container_name, sha256, reason, state	Учёт изолированных объектов.
exclusions	path, recursive, created_by	Пути, которые не сканируются автоматически.
schedules	scan_type, target, days_mask, time, enabled	Периодические проверки.
settings	key, value	Параметры приложения.
update_history	from_version, to_version, package_hash, status	История обновления базы.

Небольшие параметры интерфейса можно хранить в JSON либо через QSettings. Однако защищённые настройки, влияющие на безопасность (например, системные исключения), лучше хранить централизованно через ConfigManager и журналировать изменение. Каталог карантина остаётся

файловым хранилищем; SQLite содержит только связь контейнера с исходным объектом.

11 Обновление базы сигнатур

UpdateManager получает пакет из заранее настроенного локального или сетевого источника. Для учебной версии пакет может представлять ZIP/SSUPD-архив с manifest.json и новой SQLite-базой либо набором записей. Манифест содержит format_version, signature_version, created_at и ожидаемый SHA-256 полезной нагрузки.

1) Скачать или скопировать пакет во временный файл, не затрагивая рабочую базу.
2) Считать манифест и проверить, что версия пакета новее установленной и формат поддерживается.
3) Вычислить SHA-256 полезной нагрузки и сравнить с ожидаемым значением.
4) Распаковать данные во временный каталог, открыть новую базу read-only и проверить обязательные таблицы и schema_version.
5) Остановить новые сигнатурные запросы, дождаться завершения активных транзакций и закрыть рабочие соединения.
6) Сохранить резервную копию предыдущей базы и атомарно заменить рабочий файл.
7) Открыть новую базу, выполнить тестовый запрос и только после этого зафиксировать UPDATE_OK. При ошибке восстановить резервную копию.

Проверка SHA-256 защищает от случайного повреждения пакета. В промышленной системе дополнительно потребовалась бы криптографическая подпись обновления и полноценная модель доверия к серверу обновлений. В версии 1.0 источник считается доверенным согласно ТЗ.

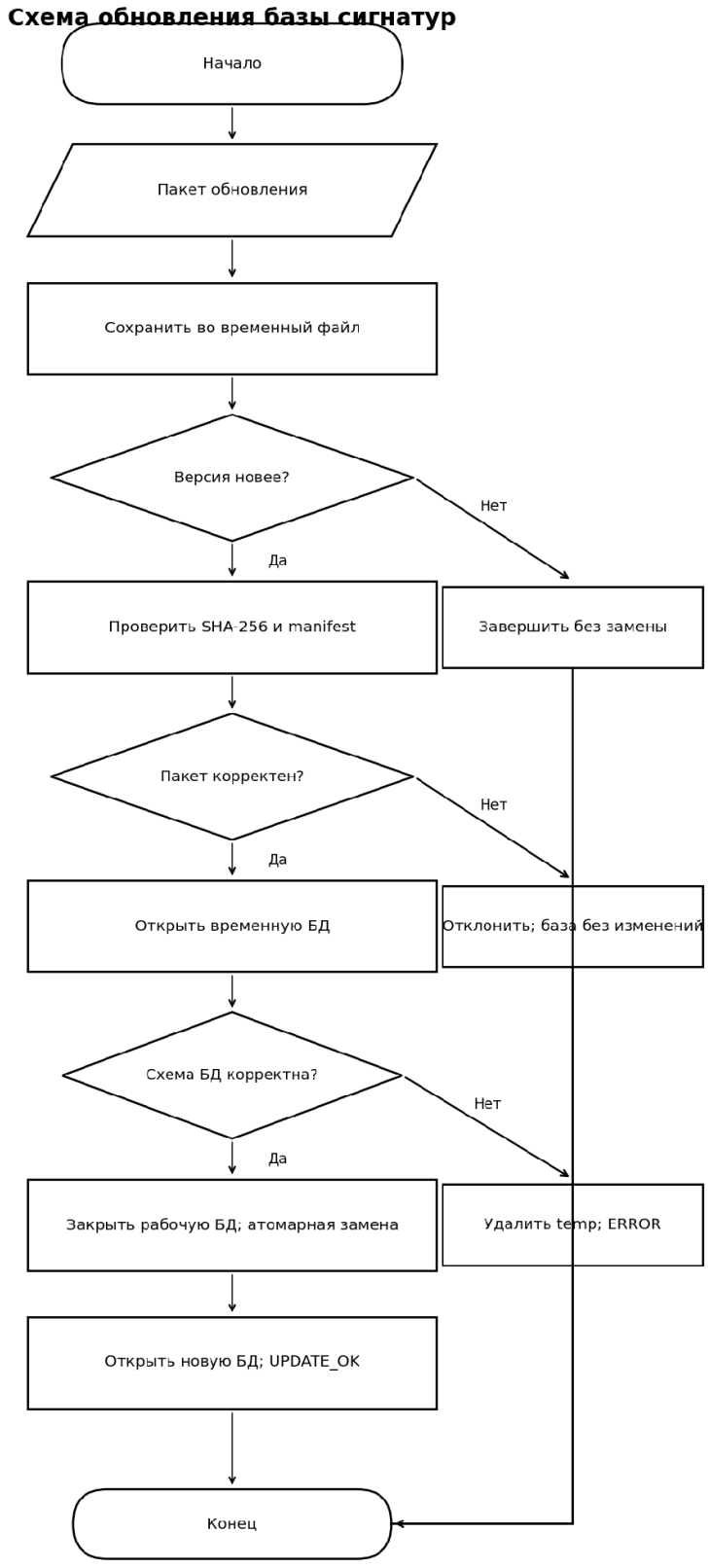


Рисунок 9 — Блок-схема обновления сигнатурной базы по ГОСТ 19.701-90

12 Планировщик проверок

SchedulerManager хранит расписание в таблице schedules. Для лабораторной версии нет необходимости создавать отдельную Windows-службу. Пока приложение запущено, таймер Qt раз в минуту проверяет, какие задания должны стартовать. При запуске приложения дополнительно проверяются пропущенные задания: если запланированная проверка не была выполнена из-за выключенного компьютера, она может быть запущена один раз после старта в соответствии с параметром run_missed.

Каждая запись расписания содержит тип проверки, путь или профиль, дни недели, время, признак enabled и last_run. Scheduler не запускает Scanner напрямую, а создаёт обычный ScanRequest в ScanManager. Поэтому ручные и плановые проверки проходят по одному и тому же коду и одинаково журналируются.

13 Пользовательский интерфейс

Для GUI выбирается Qt Widgets, так как интерфейс преимущественно состоит из форм, таблиц и списков. Интерфейс не должен скрывать важные решения антивируса: пользователь видит актуальность базы, статус мониторинга, последнюю проверку и число объектов в карантине.

Экран	Основные элементы
Главная	Состояние защиты, версия базы, мониторинг, последняя проверка, кнопки быстрой/полной/выборочной проверки.
Проверка	Текущий путь, обработано файлов, обнаружения, ошибки, прогресс, кнопка «Остановить».
Карантин	Исходный путь, дата, SHA-256, причина, восстановление, удаление из карантина по подтверждению.
Журнал	Фильтры по времени/типу/результату, подробности события, экспорт CSV/JSON.
Обновление	Текущая версия базы, источник, проверка обновлений, установка локального пакета.
Расписание	Список заданий, тип проверки, путь, дни, время, включено/выключено.
Настройки	Пути мониторинга, лимиты, политика карантина, интерфейс.

Экран	Основные элементы
Исключения	Список исключений; изменение требует административного подтверждения согласно правилам проекта.

Все сигналы из worker-потоков поступают в GUI через queued connections Qt. Нельзя изменять QWidget из фонового потока. ScanManager публикует события progressChanged, resultReady, scanFinished и scanFailed; MainWindow только отображает их.

14 Многопоточность

Проверка диска может занимать минуты, поэтому выполнение в GUI-потоке неприемлемо. Архитектура использует отдельный пул worker-потоков. Количество потоков настраивается; для учебной версии достаточно значения min(4, hardware_concurrency), чтобы не создавать избыточную нагрузку на рабочую станцию стоматологии.

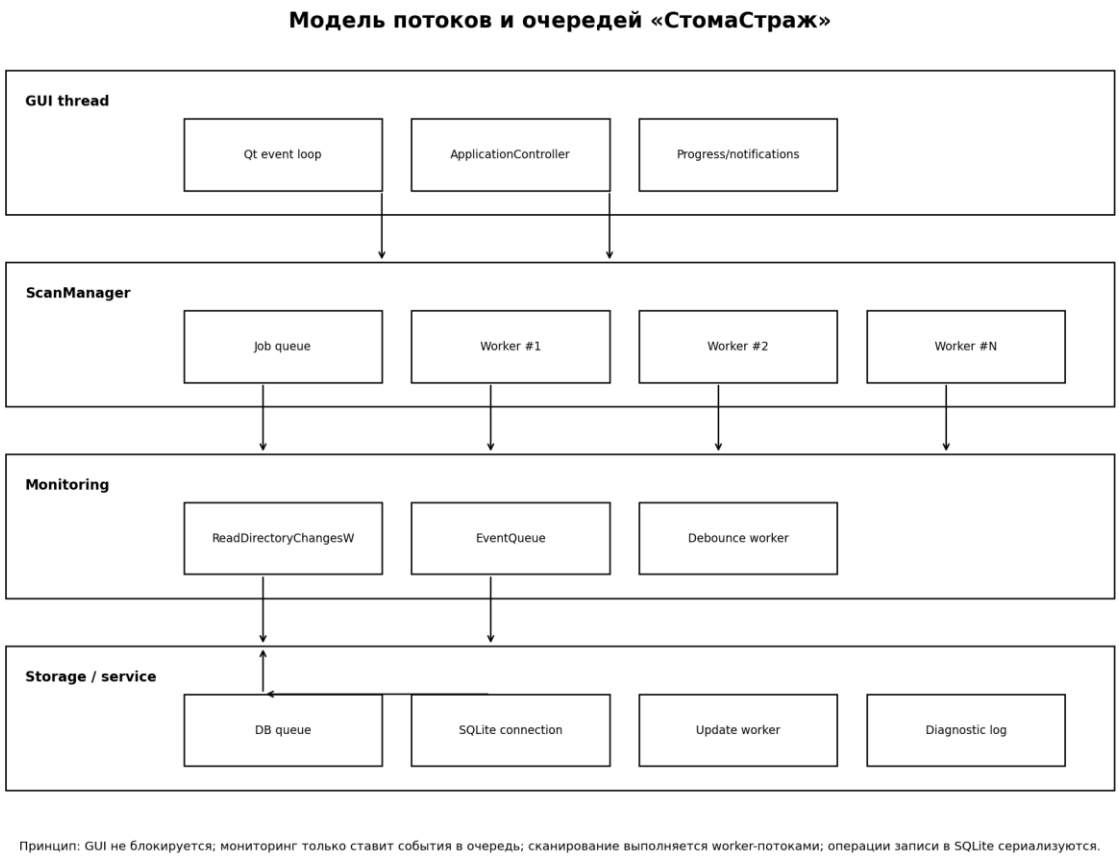


Рисунок 10 — Модель потоков и очередей

– GUI thread: event loop Qt, окна и изменение отображаемых моделей;

– ScanManager: очередь ScanJob/FileTask, распределение по worker-потокам, stop_source;

– workers: чтение файлов, SHA-256 и анализ; чистые файлы не модифицируются;

– monitor thread: получение событий Windows и постановка их в EventQueue;

– debounce worker: объединение повторных файловых событий;

– DB writer: последовательная очередь записей SQLite;

– update worker: отдельная длительная операция с эксклюзивным переключением базы сигнатур.

Общие структуры защищаются std::mutex/std::scoped_lock, ожидание очередей выполняется через std::condition_variable. Для потоков проекта

предпочтительно std::jthread, поскольку его stop_token упрощает корректную отмену и завершение приложения.

15 Классы и интерфейсы C++

Ниже приведены интерфейсы, достаточные для начала реализации. Это не окончательный код, а контракт между модулями.

```
enum class ScanStatus {
    Clean,
    Suspicious,
    Threat,
    Error,
    Skipped
};

struct ScanResult {
    std::filesystem::path path;
    ScanStatus status{ScanStatus::Error};
    std::string sha256;
    std::optional<std::string> threatName;
    int heuristicScore{0};
    std::vector<std::string> reasons;
    std::optional<std::string> error;
};

struct ScanRequest {
    std::filesystem::path target;
    bool recursive{true};
    bool allowQuarantine{true};
    std::string source; // manual / monitor / schedule
};
class Scanner {
public:
    Scanner(SignatureEngine& signatures,
            HeuristicEngine& heuristics,
            HashService& hashService,
            ExclusionManager& exclusions,
            QuarantineManager& quarantine,
            EventLogger& logger);

    ScanResult scanFile(const std::filesystem::path& path,
                        const ScanContext& context,
                        std::stop_token stopToken = {});
};
class SignatureEngine {
public:
    std::optional<ThreatSignature> lookup(std::string_view sha256) const;
    int currentVersion() const;
};

class HeuristicEngine {
public:
    HeuristicResult analyze(const FileMetadata& meta,
                            std::span<const std::byte> header) const;
};
class QuarantineManager {
public:
    QuarantineResult quarantine(const std::filesystem::path& source,
```

```
                                std::string_view expectedSha256,
                                std::string_view reason);

    RestoreResult restore(QuarantineId id,
                          const std::optional<std::filesystem::path>& newPath);
};
class FileSystemMonitor {
public:
    void addRoot(std::filesystem::path path, bool recursive);
    void removeRoot(const std::filesystem::path& path);
    void start();
    void stop();

    void setEventCallback(std::function<void(FileEvent)> callback);
};
```

Класс	Ключевые методы	Основные зависимости
ScanManager	startScan(), cancel(), enqueueFile()	Scanner, ThreadPool, EventLogger
HashService	sha256(path)	Windows CNG
SignatureEngine	lookup(), currentVersion()	DatabaseManager
HeuristicEngine	analyze()	ConfigManager
ExclusionManager	isExcluded(), add(), remove()	DatabaseManager
QuarantineManager	quarantine(), restore(), list()	DatabaseManager, HashService, EventLogger
UpdateManager	check(), installPackage()	DatabaseManager, HashService, ConfigManager
SchedulerManager	load(), dueJobs(), markRun()	DatabaseManager, ScanManager
DatabaseManager	migrate(), transaction(), repositories	sqlite3
EventLogger	event(), error(), exportLog()	DatabaseManager, file logger

16 Обработка ошибок и надёжность

Ситуация	Поведение программы
Нет доступа к файлу	Вернуть ERROR для объекта, записать код ошибки Windows, продолжить остальные файлы.
Файл удалён до чтения	Считать объект пропущенным/ошибочным; мониторинг и задание продолжаются.
Файл занят записью	Для ручной проверки показать ошибку; для мониторинга выполнить ограниченную задержанную попытку.
Файл изменился во время сканирования	Сравнить размер/mtime и назначить повторную проверку.
Повреждена база сигнатур	Не считать защиту актуальной, отключить точное обнаружение и предложить восстановление/обновление.
Ошибка SQLite	Откатить транзакцию; критические операции карантина не подтверждать как успешные.
Недостаточно места для карантина	Исходник не удалять; сообщить пользователю и записать QUARANTINE_ERROR.
Сбой пакета обновления	Отвергнуть временный пакет; рабочая база остаётся прежней.
Аварийное завершение приложения	При следующем запуске проверить записи PREPARED и временные файлы, завершить/откатить незаконченные операции.
Остановка проверки пользователем	stop_token прекращает выдачу новых задач; текущие операции завершаются безопасно; сводка сохраняется.

Важный принцип: ошибка одного файла не должна переводить всё приложение в аварийное состояние. Исключения C++ перехватываются на границе worker-задачи и превращаются в контролируемый ScanResult::Error. Исключение не должно напрямую попадать в GUI как необработанный текст.

17 Безопасность приложения

Антивирус сам работает с потенциально опасными файлами, поэтому его собственная архитектура должна уменьшать риск. Версия 1.0 строится по принципу минимальных привилегий: обычное сканирование выполняется от учётной записи пользователя, а административные права требуются только для установки, изменения защищённых системных настроек и доступа к защищённому карантину.

– файлы открываются с минимально необходимыми правами и не исполняются;
– используются Unicode API Windows и std::filesystem::path; строковая конкатенация путей не применяется;
– SQL-запросы используют параметры, а не формирование SQL из пользовательского текста;
– исключения нормализуются перед сравнением, чтобы избежать обхода через относительные компоненты пути;
– карантин имеет отдельный каталог с ограниченными ACL и случайными именами;
– в журнал не записываются содержимое файла, пароли, медицинские документы или их фрагменты;
– обновление сначала проверяется во временном каталоге и только потом заменяет рабочую базу;
– эвристика не запускает исследуемый объект и не извлекает макросы путём выполнения;

– восстановление из карантина требует явного действия и предупреждения.

Архитектура не гарантирует защиту от вредоносного кода с правами администратора, целенаправленно атакующего сам «СтомаСтраж». Самозащита промышленного уровня находится вне рамок лабораторной работы.

18 Сборка и развертывание

Основная среда разработки — Windows 10/11 x64, MSVC из Visual Studio 2022 либо совместимый компилятор, CMake и Qt 6. Сборка выполняется в x64 Release. Зависимости должны фиксироваться в README или CMakePresets.json, чтобы проект мог быть повторно собран на другом учебном компьютере.

1) cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
2) cmake --build build --config Release
3) ctest --test-dir build -C Release --output-on-failure
4) развернуть Qt runtime через windeployqt либо подготовить установочный пакет;
5) создать %ProgramData%\StomaStrazh\ с подкаталогами Data, Quarantine, Logs; применить ACL;
6) создать начальную signatures.db и default config; выполнить миграции при первом запуске.

Для учебного прототипа допустим переносимый каталог приложения, однако полноценная установка предпочтительнее, потому что позволяет корректно создать защищённый карантин и рабочие каталоги.

19 Тестирование

Архитектура должна обеспечивать выполнение ПМИ первой лабораторной без изменения её основной логики. Unit-тесты проверяют чистые функции и классы, интеграционные тесты — взаимодействие с файловой системой и SQLite, системные — готовое приложение и GUI.

Тест ПМИ	Основной модуль	Уровень
T01 Запуск	ApplicationController, DatabaseManager, UI	системный
T02 Быстрая проверка	ScanManager, Scanner	интеграционный/системный
T03 Выборочный чистый файл	HashService, Scanner	unit + интеграционный
T04 Каталог	ScanManager, filesystem	интеграционный
T05 Точная сигнатура	SignatureEngine	unit + интеграционный
T06 Эвристика	HeuristicEngine	unit
T07 Карантин	QuarantineManager	интеграционный
T08 Восстановление	QuarantineManager	интеграционный
T09 Мониторинг	FileSystemMonitor, ScanManager	интеграционный
T10 Исключения	ExclusionManager	unit + интеграционный
T11 Обновление	UpdateManager	интеграционный
T12 Экспорт	EventLogger	интеграционный
T13 Недоступный файл	Scanner	интеграционный
T14 Удаление во время проверки	Monitor/Scanner	интеграционный
T15 Повреждённое обновление	UpdateManager	интеграционный
T16 Перезапуск	DatabaseManager, QuarantineManager	системный
T17 Остановка	ScanManager	интеграционный
T18 Сохранность чистых данных	Scanner	системный
T19 Нет внешней отправки содержимого	всё приложение	системный
T20 SUSPICIOUS без автоудаления	HeuristicEngine, policy	системный

В модульных тестах реальное вредоносное ПО не используется. Сигнатурное обнаружение проверяется на безопасных файлах с заранее внесёнными SHA-256, а общая реакция антивируса — на EICAR либо внутренней безопасной имитации. Это согласуется с программой и методикой испытаний первой лабораторной.

20 Последовательность разработки

1) Создать CMake-проект, структуру каталогов, базовые типы ScanStatus/ScanResult и инфраструктуру тестов.
2) Реализовать HashService на Windows CNG и тесты SHA-256 на известных контрольных данных.
3) Реализовать SQLite-схему, миграции, DatabaseManager и репозиторий signatures.
4) Реализовать SignatureEngine и Scanner для проверки одного файла.
5) Добавить HeuristicEngine и unit-тесты каждого правила.
6) Добавить ScanManager, очередь заданий, рекурсивный обход и корректную отмену.
7) Реализовать EventLogger и историю scan_sessions/scan_items.

8) Реализовать QuarantineManager и сценарий восстановления.

9) Создать Qt GUI и связать его со ScanManager сигналами/слотами.

10) Реализовать FileSystemMonitor на ReadDirectoryChangesW с debounce.

11) Реализовать UpdateManager с временной установкой и откатом.

12) Реализовать SchedulerManager и хранение расписания.

13) Выполнить интеграционные тесты T01–T20 и устранить несогласованности документации.

Такой порядок позволяет после каждого этапа иметь работающий небольшой вертикальный срез, а не писать все подсистемы одновременно. Уже после этапа 4 можно вручную проверить файл, после этапа 8 — получить законченное ядро без GUI, а затем постепенно добавлять пользовательские функции.

21 Возможности расширения

Модульная архитектура позволяет увеличивать функциональность без изменения контрактов UI и Scanner. Возможные направления развития:

– добавление новых эвристических правил как отдельных объектов IHeuristicRule;

– поддержка сигнатур не только по полному SHA-256, но и по участкам файла, метаданным PE или шаблонам;

– выделение Scanner в фоновую Windows-службу при необходимости постоянной защиты;

– централизованное получение политик для нескольких рабочих станций;

– цифровая подпись пакетов обновлений вместо одной проверки контрольной суммы;

– добавление сетевого модуля и репутационных запросов при наличии требований по конфиденциальности;

– подключение отдельной песочницы или поведенческого анализа как внешнего сервиса, не меняя базовую модель ScanResult.

Эти функции не входят в обязательный объём версии 1.0. Их наличие в архитектуре как точек расширения не означает, что они должны быть реализованы в текущем цикле лабораторных работ.

22 Заключение

В результате лабораторной работы разработана техническая архитектура антивирусного программного обеспечения «СтомаСтраж». Она сохраняет функциональные требования первой лабораторной работы, но уточняет стек реализации в соответствии с новым требованием: основным языком становится C++20. Программа разделена на интерфейс, прикладной уровень, ядро обнаружения и инфраструктурные сервисы. Для Windows определены конкретные механизмы хеширования, мониторинга файловой системы, карантина и локального хранения данных.

Архитектура ориентирована на последовательную реализацию студентами. Первая рабочая версия не требует драйвера ядра или сложной эмуляции, но содержит ключевые свойства антивируса: точное сигнатурное обнаружение, статическую эвристику, фоновый мониторинг, карантин, журнал и обновление базы. Все обязательные функции сопоставлены с тестами T01–T20 из программы и методики испытаний.

Приложение А. Условные обозначения блок-схем

Для алгоритмических схем в работе использованы базовые условные обозначения ГОСТ 19.701-90. Стандарт определяет схемы программ как последовательность операций, соединённых линиями потока, и допускает специальные символы для облегчения чтения схемы. В настоящем документе используются следующие обозначения:

Обозначение	Назначение в схемах работы
Терминатор (фигура с закруглёнными концами)	Начало или окончание алгоритма/вход во внешнюю среду.
Процесс (прямоугольник)	Операция обработки: вычисление хеша, запись в БД, копирование файла.
Решение (ромб)	Проверка условия с ветвями «Да»/«Нет».
Данные (параллелограмм)	Входные данные или полученное событие.
Подготовка (шестиугольник)	Инициализация очереди, счётчиков и других начальных состояний.
Соединитель (круг)	Продолжение линии в другом месте схемы без длинной пересекающей линии.
Линия со стрелкой	Направление потока управления или данных.

Компонентная диаграмма, модель потоков и логическая модель базы данных не являются алгоритмическими блок-схемами; они используются как отдельные архитектурные иллюстрации и не имитируют условные обозначения ГОСТ 19.701-90.

Приложение Б. Пример CMakeLists.txt

Ниже приведён минимальный каркас сборки. Конкретные имена файлов будут уточняться по мере реализации.

```
cmake_minimum_required(VERSION 3.24)
project(StomaStrazh VERSION 1.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTOUIC ON)
set(CMAKE_AUTORCC ON)

find_package(Qt6 REQUIRED COMPONENTS Core Widgets)
find_package(SQLite3 REQUIRED)

add_library(stomastorazh_core
    src/core/Scanner.cpp
    src/core/ScanManager.cpp
    src/detection/SignatureEngine.cpp
    src/detection/HeuristicEngine.cpp
    src/security/HashService.cpp
    src/quarantine/QuarantineManager.cpp
    src/storage/DatabaseManager.cpp
    src/update/UpdateManager.cpp
    src/scheduler/SchedulerManager.cpp
    src/logging/EventLogger.cpp
    src/config/ConfigManager.cpp
)

target_include_directories(stomastorazh_core PUBLIC include)
target_link_libraries(stomastorazh_core
    PUBLIC SQLite::SQLite3
    PRIVATE bcrypt
)

add_executable(StomaStrazh
    src/main.cpp
    src/app/ApplicationController.cpp
    src/ui/MainWindow.cpp
    src/ui/ScanPage.cpp
    src/ui/QuarantinePage.cpp
    src/ui/LogPage.cpp
    src/monitoring/FileSystemMonitor_win.cpp
)

target_link_libraries(StomaStrazh
    PRIVATE stomastorazh_core Qt6::Core Qt6::Widgets
)

enable_testing()
add_subdirectory(tests)
```

Источники и нормативные документы

1. ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов.

2. ГОСТ 19.102-77. Единая система программной документации. Стадии разработки.
3. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению.
4. ГОСТ 19.301-79. Единая система программной документации. Программа и методика испытаний.
5. ГОСТ 19.404-79. Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению.
6. ГОСТ 19.701-90 (ИСО 5807-85). Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения.
7. Комплект программной документации «СтомаСтраж», лабораторная работа № 1: техническое задание, руководство оператора, руководство программиста/системного программиста, пояснительная записка, программа и методика испытаний.